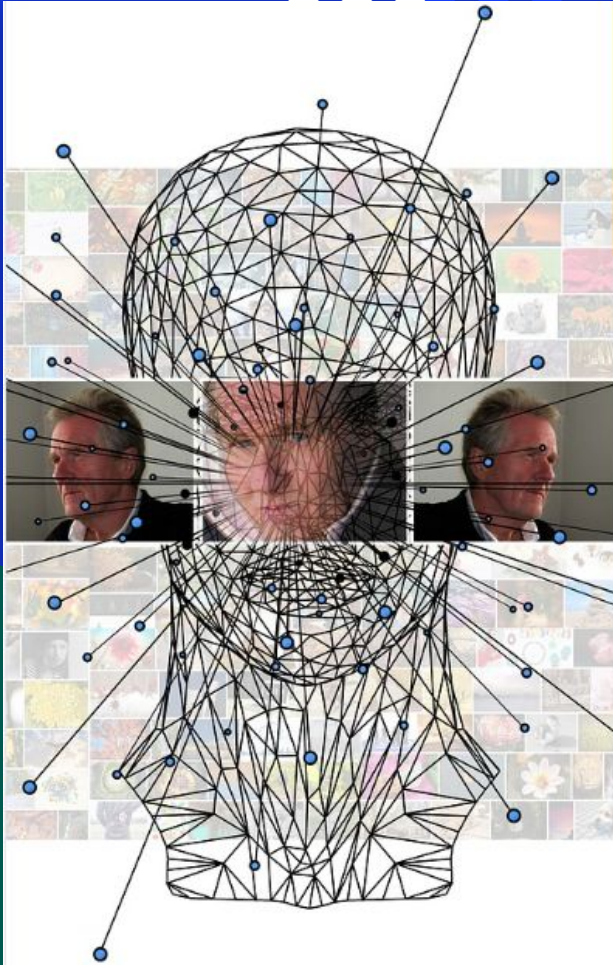




Fundamentals of Machine Learning: Classification and Clustering Techniques

An In-Depth Exploration of K-Nearest Neighbors, K-Means, and Hierarchical Agglomerative Clustering

Prepared by: Raphael Martin, Julien Klinger

Presentation Agenda

- 1) Motivation and Definitions
 - Motivation and definitions of key clustering and classification methods
- 2) K-Nearest Neighbors (KNN)
 - Theory and implementation
- 3) K-Means Clustering
 - Principles and coding walkthrough
- 4) Hierarchical Agglomerative Clustering (CAH)
 - Concepts and dendrograms
- 5) Code Examples and Visualizations
 - Code examples and visualizations for each method
- 6) Applications and Evaluation
 - Applications, advantages, disadvantages, and evaluation metrics



Why Use These Methods?

1) Simplicity and Interpretability

- These algorithms are simple, interpretable, and widely applied across various domains.

2) Supervised Classification with KNN

- KNN is ideal for supervised classification tasks.

3) Prototyping and Foundational Models

- They serve as excellent tools for prototyping, building foundational models, and teaching core machine learning concepts.

4) Clustering Techniques

- K-Means is used for partition-based clustering. CAH is effective for hierarchical clustering and exploratory data analysis.



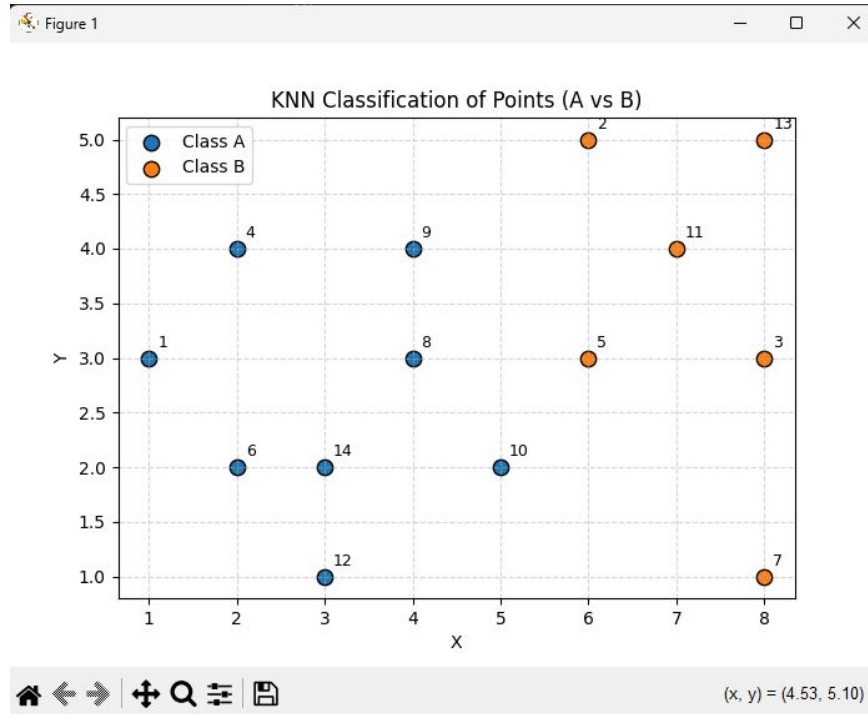
KNN Theory: Key Concepts

- 1) Overview of K-Nearest Neighbors
 - K-Nearest Neighbors is an instance-based, lazy learning method used for classification or regression.
 - It assigns the class of a new data point based on a majority vote among its k closest neighbors.
- 2) Advantages of KNN
 - Simple to implement and effective on small datasets.
- 3) Key Characteristics of KNN
 - Non-parametric approach, requiring no explicit training phase.
 - Sensitive to the scale of input variables, so normalization is important.
- 4) Challenges with KNN
 - Computational cost grows with dataset size, especially at inference.

KNN Code Example

```
def calculate_distances (target_point, df):  
    # Find distances from target to all labeled points  
    target = df[df["point"] == target_point][["X", "Y"]].values[0]  
    categorized = df[df["category"].notna()]  
    results = []  
  
    for _, row in categorized.iterrows():  
        other_point = (row["X"], row["Y"])  
        dist = math.sqrt(  
            (target[0] - other_point[0]) ** 2 + (target[1] - other_point[1]) ** 2  
        )  
        results.append((row["point"], row["category"], dist))  
  
    return results
```

KNN Visualization



K-Means Theory: Core Principles

1) Iterative Process of K-Means

- Assign points to the nearest centroid
- Recalculate centroids based on current cluster members
- Repeat until cluster assignments stabilize (convergence)

2) Key Considerations in K-Means

- Sensitive to initial centroid placement and choice of k
- Best suited for spherical, well-separated clusters

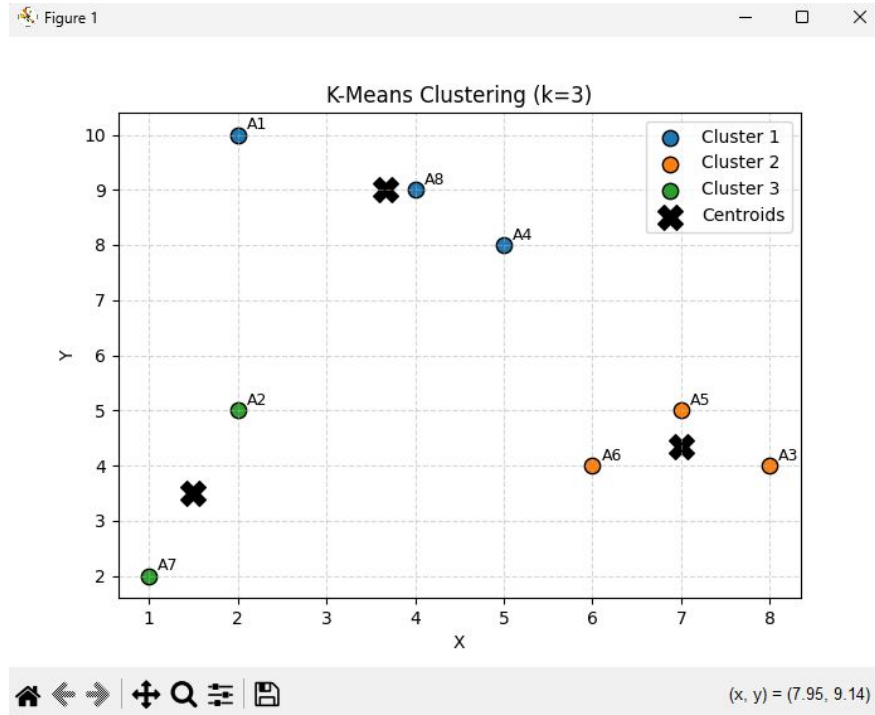
3) Core Principle of K-Means

- Partitioning clustering algorithm that divides data into k groups
- Minimizes the variance within each cluster

K-Means Code Example

```
def compute_centroids(X, clusters, k):  
    new_centroids = []  
    for i in range(k):  
        points_in_cluster = X[clusters == i]  
        if len(points_in_cluster) > 0:  
            new_centroids.append(points_in_cluster.mean(axis=0))  
        else:  
            new_centroids.append(random.choice(X))  
    return np.array(new_centroids)
```

K-Means Visualization



Hierarchical Agglomerative Clustering (CAH) Theory

Dendrogram Representation

CAH builds a nested hierarchy of clusters represented as a dendrogram.

Agglomerative Approach

It uses an agglomerative approach: starting with individual points, It progressively merges the closest clusters.

Linkage Criteria

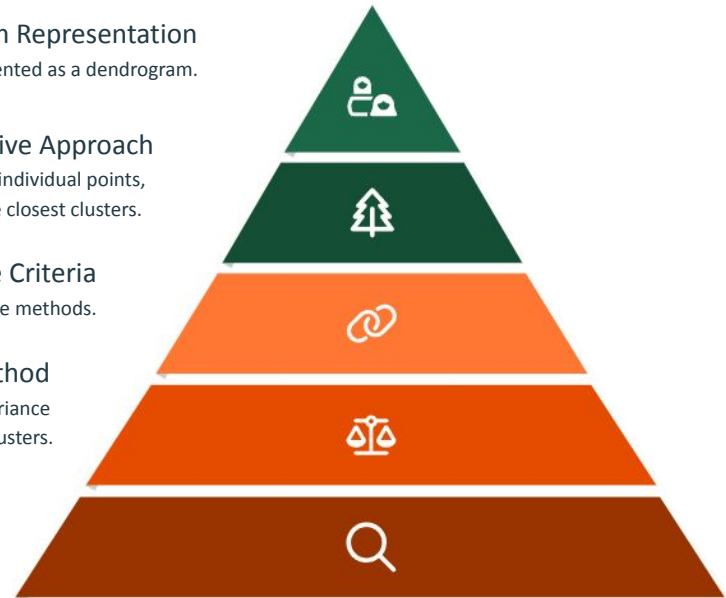
Linkage criteria include Ward, single, and complete linkage methods.

Ward's Method

Ward's method specifically minimizes the increase in intra-cluster variance at each merge, producing compact clusters.

Exploratory Data Analysis

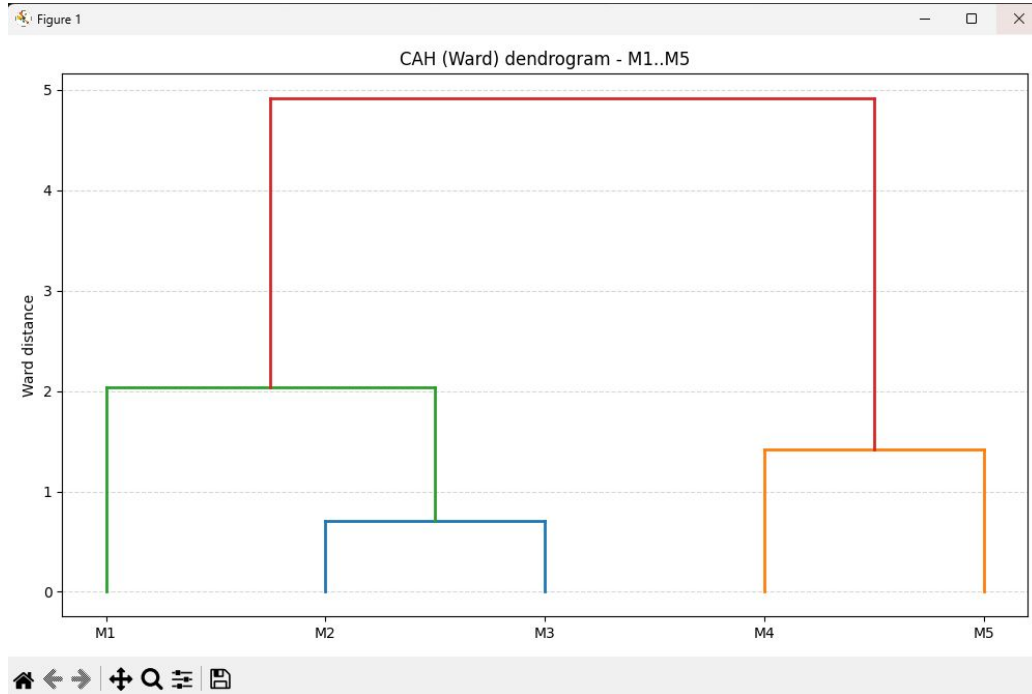
This approach is valuable for exploratory data analysis and visualizing cluster relationships.



CAH Code Example with Ward Linkage

```
def ward_distance(cluster1, cluster2):  
    centroid1 = np.mean(cluster1, axis=0)  
    centroid2 = np.mean(cluster2, axis=0)  
    n1, n2 = len(cluster1), len(cluster2)  
    return np.sqrt(  
        (n1 * n2) / (n1 + n2) * euclidean_distance(centroid1, centroid2) ** 2  
    )
```

CAH Visualization



Method Comparison and Choosing the Right One

KNN

- **Supervised classification, small datasets**
- **Simple, non-parametric, interpretable**
- **Slow inference, needs normalization, sensitive to k**

K-Means

- **Unsupervised clustering, segmentation**
- **Fast, easy to implement, no need for k in some variants**
- **Sensitive to initialization and k, assumes spherical clusters**

Evaluation Metrics and Best Practices

- 1) Importance of Normalization in Distance-Based Methods
 - Always normalize variables to ensure fair comparisons
- 2) Cluster Assessment for K-Means and CAH
 - Silhouette score to assess cluster cohesion and separation
 - Davies–Bouldin and Calinski–Harabasz indices for cluster validity
- 3) Confusion Matrix Analysis
 - Used to evaluate the performance of classification models
 - Provides insights into true positives, false positives, true negatives, and false negatives
- 4) ROC-AUC for Binary Classification
 - Measures the performance of binary classification models
 - Evaluates the trade-off between sensitivity and specificity

Real-World Applications

- 1) KNN: Similarity Search and Recommendation Systems
 - Similarity search and recommendation systems, including approximate nearest neighbors (ANN)
- 2) K-Means: Versatile Applications
 - Customer segmentation
 - Image compression
 - Anomaly detection in various domains
- 3) CAH: Hierarchical and Analytical Uses
 - Genomic data analysis
 - Document classification
 - Building hierarchical taxonomies



Questions